QUADRASIM – AN ADAPTIVE SPATIAL PARTITIONING ENGINE FOR REAL-TIME COLLISION DETECTION PROJECT PROPOSAL

Adam Ben-Aamr

adam.benaamr@uri.edu

The University of Rhode Island

CSC 212: Data Structures and Abstractions

Project Overview

Team Members

- Adam Ben-Aamr

TA or Instructor Consultation

- Marco Alverez on October 14, 2026

GitHub Repository

- https://github.com/adambenaamr/QuadraSim

Problem and Motivation

Collision detection in real time with a large number of moving objects is challenging. Comparing each object with each of the others is a possible solution. However, this brute-force approach requires a time complexity of $O(n^2)$, where n is the number of objects. The simulation runs become slower with an increase in the number of objects. Applications like physics and video games that need many spatial queries can be severely limited.

This project uses spatial partitioning to speed up collision detection. In particular, this project will use and measure the performance of Quadtree. A Quadtree is a data structure used for recursive subdivision of two-dimensional space. The project will measure and compare the performance of Quadtree against a brute-force approach and a uniform spatial grid.

An interesting problem of using spatial partitioning is how different spatial partitioning structures perform depending on the particle distribution. Although Quadtree can reduce unnecessary checks, a lot of overhead is introduced by having a structure. The simulation particle count and distribution as well as Quadtree parameters need to be examined.

Proposed Solution and Data Structure

The program will let you run an interactive 2D simulation to test and measure how good different spatial partitioning algorithms are at detecting collisions between particles. The particles will be able to move inside the borders of the world and collide and bounce off the borders of the world. In C++, there will be three methods of collision detection that can be run against each other. Those methods will be brute force pairwise, a uniform spatial grid, and a Quadtree. Each of those methods will be tested against different particle counts, different spatial configurations, and different algorithm configurations.

For the primary data structure of the Quadtree, each node in the tree will be a two-dimensional spatial area. Each node in the tree will be a data structure that will store the information about all the particles inside it and the information about the descendants of that node. A particle system will contain many regions, and when a node in the Quadtree has too many particles in its region, that region will be partitioned into four quadrants of equal area. Each quadrant will have the potential of becoming a new node in the Quadtree. A node can have children in a hierarchical structure, and a particle can be inside more than one node in a system, but it will be inside a specific node for each level of the particle tree. With the particle system as a Quadtree, collision detection will be done in the most efficient manner as the algorithm will have the option of ignoring pairs of particles that are already separated by space, reducing the computational time required for a collision check in an efficient manner.

Quadtrees are ideal for this problem because particle locations can vary continuously and be spatially uneven. With a uniform grid, the space is divided into fixed-sized cells and so the granularity is uniform across the entire space. Unlike the uniform grid, Quadtrees adapt the space granularity based on the particle density in the space. The flexibility offered by Quadtrees can improve performance when particle density is high in some areas and low in others. It is important to note, however, that building and traversing the Quadtree introduces additional overhead. Consequently, whether it improves or hinders the performance of the simulation will depend on the simulation's attributes.

The simulation will implement both a uniform spatial grid and the Quadtree, along with a baseline brute force method which involves checking every unique pair of particles for collision.

Using C++, these methods will be implemented, and their candidate-pair counts, collision detection times, overheads for constructing indices, and their respective memory usage will be measured.

For the purposes of the project, object-oriented design and C++ modules will be used to keep particle management, collision detection, spatial indexing, simulation logic and performance measurement separate from one another. Automated testing will confirm that the implemented collision detection algorithms find the same collision pairs, and a benchmarking system will track their performance using a range of particle numbers and different particle spatial distributions. Through this project, I intend to show the utility of advanced data structures, and to determine the cases in which adaptive spatial partitioning improves performance over traditional methods.

Baseline and Evaluation Plan

The main baseline will be a brute-force collision-detection algorithm that checks every pair of particles to see if a collision has happened. This method needs $O(n^2)$ comparisons for n particles. Will give a reference point for measuring how much faster spatial partitioning can be. The Quadtree approach will also be tested against a spatial grid, which splits the simulation area into fixed-size cells and limits collision checks to nearby cells. All three algorithms will use the collision-detection rules and particle settings so that the comparison is fair.

Testing will use generated particle placements inside a bounded two-dimensional space. The placements will cover random positions, clusters, very dense spots and sparse spreads. The number of particles will grow step by step: 100 first then 500, 1,000, 2,500, 5,000, 10,000, and maybe 25,000 or more depending on how the hardware handles it. Extra tests will change Quadtree settings, such as how many particles a node can hold and how deep the tree can grow, to see how the setup changes speed.

The main measurements will be the time taken for collision detection the number of candidate particle pairs that were checked the time to build the index and the amount of memory used. The Quadtree will also record how nodes it has and its deepest level so we can see how much overhead the hierarchy adds. Every algorithm will run with the particle setups and we will repeat each test many times to lower the effect of random noise. We will confirm correctness by comparing the collision pairs found by each method with those found by the brute-force baseline.

Results will be shown with tables and graphs that compare how fast each algorithm is across numbers of particles different spatial layouts and different settings. Especially plots of time and candidate-pair counts, versus particle count will show how each method scales when the workload grows. We will also compare how the particle layout and the Quadtree setup affect speed. Finally, we will analyze the data against what theory predicts for each algorithm so we can spot when spatial partitioning gives the advantage and when its extra cost makes it less useful.

Planned Demonstration

The finished project will be a two-dimensional particle simulation written in C++. This particle simulation will show how spatial partitioning can make collision-detection faster. Particles will move inside a space collide with each other and bounce off the walls. Users can change how particles there are change their spatial distribution, pause or resume the particle simulation and choose between three collision-detection algorithms: brute force, a uniform spatial grid and a Quadtree.

The application will show a real-time visualization of the Quadtree's subdivisions. This visualization will let users see how the environment is split into regions as particle density grows. When a collision query runs the application will highlight the Quadtree regions and the candidate particle pairs. This makes it clear how the Quadtree limits the search to particles instead of checking every pair. A performance dashboard will show metrics like candidate-pair counts, collision-detection time, index-construction time and the number of Quadtree nodes.

During the demonstration I will start with a small number of particles and show how each collision-detection algorithm finds collisions. Then I will gradually increase the particle count to show the cost of brute-force collision detection compared to the spatial partitioning methods. Next I will present both uniform and clustered particle distributions to illustrate how workload patterns affect the Quadtree's performance versus the uniform grid. Finally, I will run the application's benchmarking mode with particle setups and display the resulting performance measurements.

The main goal of the demonstration is to make the Quadtree's advantage clear both visually and in numbers. The spatial subdivision overlay will show how the Quadtree organizes the environment. At the time the performance dashboard will reveal whether this organization cuts down on needless collision checks and speeds up execution. The demonstration will also point out cases where building

and walking through the Quadtree adds work or where dense clusters reduce its usefulness. By mixing visualization, with measurable performance data the finished project will show not only how the Quadtree works but also when and why it can make collision detection more efficient.